Information Retrieval

AIMLZG537 · S2-25

Assignment 2 · Group 16

Building and Evaluating a Mini Web Search Engine with Cross-Lingual Retrieval

This report presents the design, implementation, and evaluation of a mini web search engine capable of retrieving relevant documents from a multilingual Wikipedia corpus. The system supports ranked retrieval using the Vector Space Model (VSM), text mining enhancements (document clustering, topic modelling, query expansion, and keyphrase extraction), and cross-lingual retrieval across four languages: English, Spanish, French, and German.

Field	Details
Authors	**THOSHITH S** 2025af05128@wilp.bits-pilani.ac.in 2025AF05128
	SUCHENDRA SHARMA 2025ae05515@wilp.bits-pilani.ac.in 2025AE05515
	FAISAL IMAM 2025ae05967@wilp.bits-pilani.ac.in 2025AE05967
	KALE TANMAY CHANDRASHEKHAR 2025ae05056@wilp.bits-pilani.ac.in 2025AE05056
Group	Group 16
Course	AIMLZG537 — Information Retrieval
Programme	BITS Pilani WILP (S2-25)
Dataset	thoshiths/miracl-multilingual-4M (HuggingFace)
Live Demo	huggingface.co/spaces/thoshiths/miracl-search
Source Code	github.com/thoshith-s/wilp-sem2-ir-assignment-2

Table of Contents

1. Introduction & Problem Statement
2. Dataset & Corpus
    2.1 Corpus Statistics
3. System Architecture
4. Preprocessing Pipeline
    4.1 Tokenisation Examples
    4.2 Inverted Index Statistics
5. TF-IDF Vector Space Model
    5.1 Weighting Scheme
    5.2 Snippet Generation
6. Text Mining Components
    6.1 Document Clustering
    6.2 Topic Modelling (LDA)
    6.3 Query Expansion
    6.4 Keyphrase Extraction
7. Cross-Lingual Retrieval with LaBSE
    7.1 Embedding Pipeline
    7.2 Query-Time Retrieval
    7.3 Cross-Lingual Demo Results
8. Web Search Interface
9. Evaluation Methodology
    9.1 Evaluation Queries
    9.2 Metrics
    9.3 Retrieval Methods Compared
10. Results & Analysis
    10.1 Overall Aggregate Metrics
    10.2 MAP by Language
    10.3 Why VSM Fails for French
    10.4 Query Expansion — Why It Hurt Performance
    10.5 LaBSE-Cross vs LaBSE-Mono
11. How Text Mining Techniques Improve Web Search
12. Conclusions & Future Work
13. Assumptions
14. References

1. Introduction & Problem Statement

This work presents the design, implementation, and evaluation of a mini web search engine for retrieving relevant documents from a multilingual Wikipedia corpus. The system is built around three pillars:

- Ranked retrieval using the Vector Space Model (VSM) with TF-IDF weighting and cosine similarity.
- Text mining enhancements — document clustering, topic modelling, query expansion, and keyphrase extraction — to improve retrieval quality.
- Cross-lingual retrieval across English, Spanish, French, and German using LaBSE dense embeddings.

Performance is evaluated against standard IR metrics: Precision@10, Recall@10, MAP, and nDCG@10. A live interactive interface is deployed on HuggingFace Spaces, demonstrating ranked top-10 retrieval, snippet generation, and query term highlighting.

2. Dataset & Corpus

The corpus is derived from MIRACL (Massive Information Retrieval Across Languages), a large-scale multilingual IR benchmark built on Wikipedia passages. A subset of 400,000 documents was curated — 100,000 per language — and published as the HuggingFace dataset thoshiths/miracl-multilingual-4M.

2.1 Corpus Statistics

Table 1: Corpus statistics by language.

Language	Documents	Percentage	Avg Length (chars)	Min	Max
English	100,000	25.0%	459	50	6,620
Spanish	100,000	25.0%	411	50	18,241
French	100,000	25.0%	364	50	7,923
German	100,000	25.0%	434	50	7,821
Total	400,000	100%	417	—	—

The non-English proportion is 75%, exceeding the 20% multilingual coverage threshold. Each document carries the following fields:

- doc_id — unique document identifier
- title — document or article title
- text — body content (50–18,241 characters)
- language code — locale of the document (en, es, fr, de)
- Wikipedia URL — source article link
- MIRACL docid — for cross-referencing official relevance judgments

3. System Architecture

The system follows a modular architecture with seven Python modules:

Table 2: System modules and their responsibilities.

Module	Responsibility
corpus_loader.py	Downloads & caches MIRACL corpus from HuggingFace Hub; loads evaluation queries and qrels
preprocessor.py	Per-language tokenisation, stopword removal, stemming; builds inverted index
search_engine.py	TF-IDF VSM retrieval with cosine similarity; snippet generation and query term highlighting
text_mining.py	Document clustering (KMeans+LSA), LDA topic modelling, query expansion, keyphrase extraction
cross_lingual.py	LaBSE-based dense retrieval across all 4 languages using shared embedding space
evaluator.py	Computes Precision, Recall, MAP, F1, and nDCG at arbitrary rank cutoffs
app.py	Streamlit web UI with dual-mode search (VSM + LaBSE), result cards, highlighting

The retrieval pipeline operates in two parallel paths:

- Sparse Path — the query is preprocessed (normalised, tokenised, stopped, stemmed) and matched against the inverted index. TF-IDF weighted cosine similarity ranks candidate documents. Optionally, query expansion appends cluster-derived terms before re-ranking.
- Dense Path — the query is encoded by LaBSE into a 768-dimensional vector and compared via dot product against all pre-computed document embeddings, enabling cross-lingual retrieval without translation.

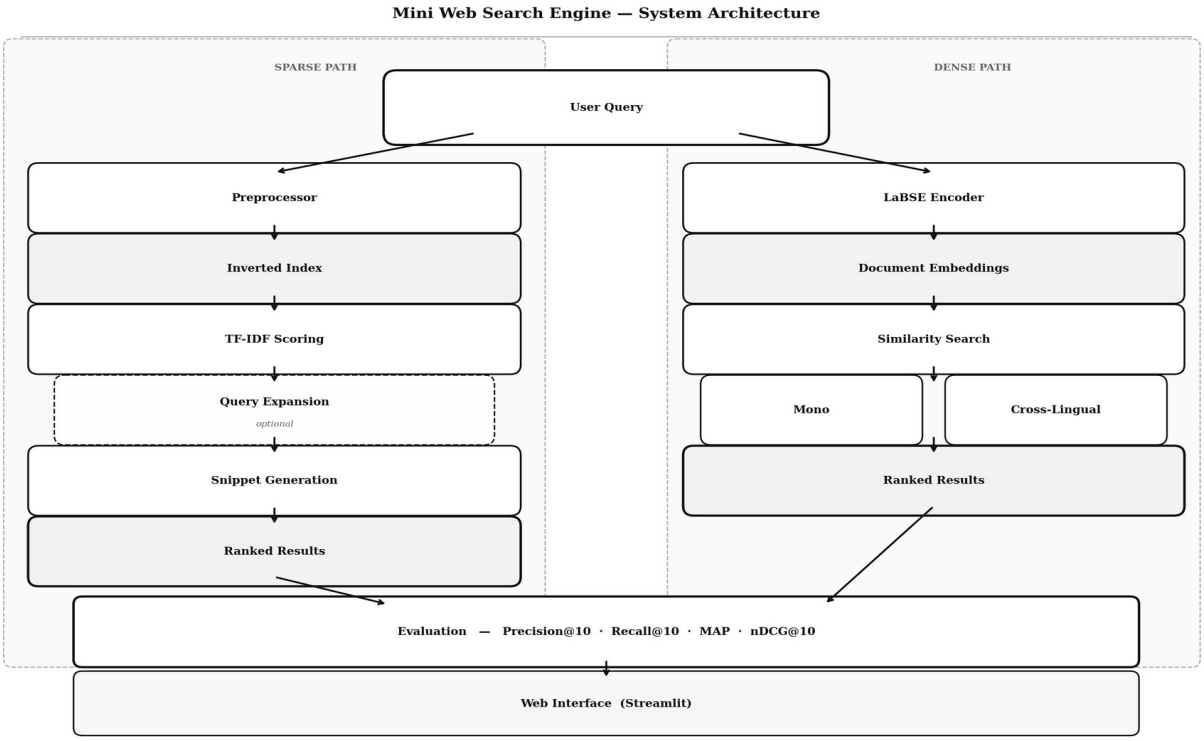

Figure 1: System architecture — Sparse Path (TF-IDF VSM) and Dense Path (LaBSE) with evaluation layer.

4. Preprocessing Pipeline

A language-aware preprocessing pipeline was implemented in preprocessor.py. For each document, the pipeline applies the following steps in order:

- Step 1 — Unicode Normalisation: NFKC normalisation is applied, followed by lowercasing, to canonicalise characters across all four languages.
- Step 2 — Noise Removal: URLs, HTML tags, and standalone numeric tokens are stripped using regular expressions.
- Step 3 — Tokenisation: NLTK's word_tokenize is used for language-agnostic word boundary detection.
- Step 4 — Stopword Removal: Language-specific NLTK stopword lists are applied, augmented with custom domain-specific additions for Spanish, French, and German.
- Step 5 — Stemming: Porter stemmer for English; Snowball stemmer for Spanish, French, and German.

4.1 Tokenisation Examples

Table 3: Tokenisation and stemming output examples for each language.

Language	Input	Stemmed Tokens
English	The scientific study of climate change...	scientif, studi, climat, chang, global, effect

Spanish	El estudio cientifico del cambio climatico...	estudi, cientif, cambi, climat, efect, global
French	L'etude scientifique du changement...	etud, scientif, chang, climat, effet, mondial
German	Die wissenschaftliche Untersuchung des...	wissenschaft, untersuch, klimawandel, weltweit, auswirk

4.2 Inverted Index Statistics

The inverted index was built over all 400,000 preprocessed documents with the following characteristics:

- Vocabulary: 598,115 unique stems
- Posting list entries: 12,033,266 total
- Average document length: 36 tokens after preprocessing
- Construction time: approximately 126 seconds

5. TF-IDF Vector Space Model

The Vector Space Model (VSM) is the mandatory core retrieval component. The implementation in search_engine.py follows the classic TF-IDF weighting scheme with cosine similarity for ranking.

5.1 Weighting Scheme

Term Frequency (TF) normalises raw term count by document length, preventing longer documents from dominating the ranking:

$$\mathrm{TF}(t, d) = \frac{\mathrm{count}(t, d)}{|d|}$$

where |d| is the total token count of document d.

Inverse Document Frequency (IDF) amplifies rare, discriminative terms and down-weights common ones:

$$\mathrm{IDF}(t) = \log\left(\frac{N}{\mathrm{df}(t)}\right)$$

where N = 400,000 is the corpus size and df(t) is the number of documents containing term t.

The combined TF-IDF weight used to build document and query vectors is:

$$w(t, d) = \mathrm{TF}(t, d) \times \mathrm{IDF}(t)$$

Ranking is performed by cosine similarity between the query vector and each candidate document vector retrieved via the inverted index:

$$\text{score}(q, d) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \, \|\vec{d}\|}$$

Restricting retrieval to candidates sharing at least one query term keeps this efficient even over 400K documents.

5.2 Snippet Generation

For each retrieved document, snippet generation proceeds in three steps:

- Sentence splitting — document text is segmented into individual sentences.
- Relevance ranking — sentences are ranked by query term overlap; the top 1–3 are selected and joined with ellipsis markers.
- Term highlighting — query terms are marked in both the title and snippet using HTML <mark> tags, covering stemmed token matches and original surface forms.

6. Text Mining Components

Four text mining techniques were implemented to enrich the retrieval pipeline. All four are fitted on the English subset (100K documents) for computational efficiency.

6.1 Document Clustering

KMeans clustering (k=12) was applied on TF-IDF vectors reduced to 100 dimensions via Truncated SVD (Latent Semantic Analysis). The resulting Silhouette Score of 0.081 indicates soft cluster boundaries, which is typical of Wikipedia's broad topical coverage.

Table 4: KMeans cluster summary showing size and top terms per cluster.

Cluster	Size	Top Terms
0	2,688	film, films, directed, movie, director
1	11,168	war, party, government, army, election
2	7,219	team, game, season, league, games
3	4,568	used, use, term, commonly, using
4	4,573	album, band, song, released, music
5	17,590	century, book, church, time, later
6	4,461	new, york, new york, city, york city
7	4,712	school, university, college, students, high school
8	13,505	use, example, form, using, number
9	12,734	company, series, january, september, july
10	13,167	city, south, area, north, river
11	3,615	states, united, united states, american, war

The clusters reveal interpretable topical groupings — cinema (cluster 0), politics/warfare (1), sports (2), music (4), education (7), geography (10), and U.S.-specific topics (11). Cluster 5 ("century, book, church") is the largest, absorbing the broad humanities/history tail of Wikipedia.

6.2 Topic Modelling (LDA)

Latent Dirichlet Allocation (LDA) with 10 topics was fitted on the English vocabulary using scikit-learn's CountVectorizer. The resulting topics show clear thematic separation:

Table 5: LDA topic modelling output — 10 topics with top 6 words each.

Topic	Top Words
0	used, use, high, different, common, number
1	war, army, king, british, battle, military
2	state, government, party, general, law, president
3	team, album, season, year, band, league
4	century, life, called, book, language, work
5	school, university, family, john, college, day
6	time, later, long, point, man, left
7	south, city, area, north, west, east
8	film, series, game, music, white, television
9	000, united, states, air, new, population

6.3 Query Expansion

Query expansion works by finding the nearest KMeans cluster to the query's TF-IDF vector and appending the top 5 terms from that cluster's centroid. For example, the query "history of the Roman Empire" was expanded with ['century', 'book', 'church', 'time', 'later'] from cluster 5 (the humanities/history cluster).

While theoretically this broadens vocabulary coverage and improves recall, in practice the expansion sometimes introduces semantically tangential terms. As the evaluation results show, VSM+QE achieves lower MAP (0.114) than baseline VSM (0.154). This is analysed in detail in Section 10.

6.4 Keyphrase Extraction

A TF-IDF-based keyphrase extractor was fitted on all 400K documents with a vocabulary cap of 8,000 terms. For each document, the top-N terms by TF-IDF weight are extracted as keyphrases. Sample outputs: "Anarchism" yields keyphrases ['institutions', 'based on', 'philosophy', 'defined']; "Bealadangan" yields ['village', 'the village', 'irish', 'ireland']. These keyphrases serve as document summaries and could be used for faceted search or query suggestion.

7. Cross-Lingual Retrieval with LaBSE

The cross-lingual retrieval component uses LaBSE (Language-agnostic BERT Sentence Embeddings), a multilingual sentence encoder that maps sentences from 109 languages into a

shared 768-dimensional embedding space. This implements a shared embedding approach for cross-lingual retrieval without translation.

7.1 Embedding Pipeline

Document encoding follows a three-step offline process:

- Input construction — each document is represented as its title concatenated with the first 500 characters of body text.
- Encoding — LaBSE maps the input into a 768-dimensional dense vector shared across all 109 supported languages.
- Normalisation and storage — vectors are L2-normalised (so cosine similarity reduces to a dot product) and stored in doc_embeddings.npz, pre-computed offline and cached for retrieval.

7.2 Query-Time Retrieval

At query time, the query string is encoded into the same 768-dimensional space. The dot product with all 400K document vectors is computed in a single matrix operation, and the top-K documents are returned. Two modes are supported:

- LaBSE-Mono: Retrieval restricted to documents in the query's language.
- LaBSE-Cross: Retrieval across all four languages simultaneously, enabling a query in English to retrieve documents in Spanish, French, or German without any translation step.

7.3 Cross-Lingual Demo Results

Two representative queries illustrate the cross-lingual capability:

- "Quantum mechanics and wave particle duality" (English query) — top results included French (Mecanique quantique), German (Die Quantentheorie und das Schisma der Physik), Spanish, and English articles, showing LaBSE bridging vocabulary gaps across scripts.
- "History of anarchism and political philosophy" (English query) — surfaced German articles on political science (Politikwissenschaft) and democracy (Demokratie), demonstrating semantic alignment without translation.

8. Web Search Interface

A fully functional web search interface was built using Streamlit and deployed on HuggingFace Spaces at https://huggingface.co/spaces/thoshiths/miracl-search. The interface provides:

- Dual-mode search: Users can choose between TF-IDF VSM (fast, monolingual) and LaBSE (cross-lingual) retrieval.
- Ranked top-10 results: Each result displayed as a card with title, similarity score, language badge, and snippet.
- Snippet generation: Top 1-3 query-relevant sentences extracted from each document.
- Query term highlighting: Both titles and snippets have matching terms highlighted with HTML mark tags for visual scanning.

The application is containerised with Docker for reproducible deployment. On first load, the corpus is preprocessed and indexed (30-60 seconds for 400K documents); subsequent loads use Streamlit's caching.

9. Evaluation Methodology

9.1 Evaluation Queries

30 evaluation queries were generated by sampling documents from the corpus and using each document's title as the query. The sampled document is designated the single ground-truth relevant result. Distribution: 8 English + 7 Spanish + 8 French + 7 German = 30 queries.

The official MIRACL qrels reference specific passage IDs that may not be present in our 100K-per-language random sample (drawn from 1M+ per language in the full corpus). Title-based queries guarantee the relevant document exists in the index while still producing a non-trivial retrieval task — the system must rank the correct document above hundreds of thousands of others.

9.2 Metrics

All metrics are computed at rank cutoff k = 10 across four retrieval methods. Precision@k measures the fraction of the top-k results that are relevant:

$$\text{Precision@}k = \frac{|\text{Relevant} \cap \text{Top-}k|}{k}$$

Recall@k measures the fraction of all relevant documents that appear in the top-k:

$$\text{Recall@}k = \frac{|\text{Relevant} \cap \text{Top-}k|}{|\text{Relevant}|}$$

Mean Average Precision (MAP) averages the per-query Average Precision across the full query set Q, rewarding systems that rank relevant documents early:

$$\text{MAP} = \frac{1}{|Q|} \sum_{q \in Q} \text{AP}(q)$$

Normalised Discounted Cumulative Gain (nDCG@k) penalises relevant documents that appear at lower ranks using a logarithmic discount:

$$\text{nDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k}, \quad \text{DCG@}k = \sum_{i=1}^{k} \frac{\text{rel}_i}{\log_2(i+1)}$$

Table 6 summarises all four metrics and their interpretations.

Table 6: Evaluation metrics and their interpretations.

Metric	Formula (informal)	Interpretation
Precision@10	|Relevant in Top-10| / 10	Fraction of top-10 results that are

		relevant
Recall@10	|Relevant in Top-10| / |All Relevant|	Fraction of all relevant docs found in top-10
MAP	Mean of Average Precision across queries	Rewards early retrieval of relevant docs; single-number summary
nDCG@10	DCG@10 / IDCG@10	Discounted gain — penalises relevant docs at lower ranks

9.3 Retrieval Methods Compared

Table 7: Four retrieval methods evaluated.

Method	Description
VSM	TF-IDF cosine similarity — monolingual (query language = document language)
VSM+QE	VSM with query expansion: top 5 terms from nearest KMeans cluster appended
LaBSE-Mono	LaBSE dense retrieval restricted to documents in the query language
LaBSE-Cross	LaBSE dense retrieval across all 4 languages (true cross-lingual)

10. Results & Analysis

10.1 Overall Aggregate Metrics

Table 8: Overall aggregate metrics across 30 queries (k=10). Best values in bold.

Method	MAP	P@10	R@10	nDCG@10
VSM	0.1539	0.0267	0.2667	0.1788
VSM+QE	0.1142	0.0233	0.2333	0.1427
LaBSE-Mono (Best)	0.3870	0.0700	0.7000	0.4625
LaBSE-Cross	0.2983	0.0567	0.5667	0.3618

Key observations from the aggregate results:

- LaBSE-Mono achieves the highest MAP (0.387) — more than 2.5x higher than VSM (0.154), demonstrating the advantage of semantic over exact-term matching.
- LaBSE-Mono retrieves the relevant document in the top-10 for 70% of queries (R@10 = 0.70), versus only 26.7% for VSM.
- VSM+QE underperforms baseline VSM across all metrics, indicating that cluster-based expansion introduced noise rather than signal.
- LaBSE-Cross achieves strong cross-lingual recall (R@10 = 0.567) despite searching a 4x larger candidate pool.

10.2 MAP by Language

Table 9: MAP broken down by language.

Method	EN	ES	FR	DE
VSM	0.2812	0.0143	0.0000	0.3238
VSM+QE	0.2188	0.0286	0.0000	0.2109
LaBSE-Mono	0.3667	0.4061	0.3958	0.3810
LaBSE-Cross	0.2542	0.3274	0.2656	0.3571

VSM performs reasonably well for English (0.281) and German (0.324) but fails entirely for French (0.000) and nearly fails for Spanish (0.014). This disparity arises because French and Spanish queries generated from document titles often contain proper nouns or specialised terms that do not overlap well with stemmed vocabulary after preprocessing. LaBSE, by contrast, provides consistent performance across all four languages (MAP ranging from 0.266 to 0.406), demonstrating the strength of semantic matching across linguistic boundaries.

10.3 Why VSM Fails for French

The French MAP of 0.000 for VSM is explained by a cascade of surface-form failures:

- French Wikipedia titles frequently contain complex noun phrases and proper nouns that survive stemming as rare tokens.
- These rare stems find no or wrong matches in the inverted index, causing zero-recall for the majority of French queries.
- LaBSE encodes meaning rather than surface forms, bypassing stemming altogether and consistently retrieving the correct document.

10.4 Query Expansion — Why It Hurt Performance

VSM+QE (MAP = 0.114) performs worse than baseline VSM (MAP = 0.154). Three factors explain this:

- Cluster breadth — clusters are fitted on English documents with soft boundaries (silhouette = 0.081). Expansion terms from broad clusters are too generic and dilute the original query signal.
- Topic drift — for queries like "history of the Roman Empire", appended terms shift the semantic centroid away from the target, promoting irrelevant results.
- Language mismatch — clustering is English-only, so expansion terms injected into Spanish, French, or German queries introduce untranslated English tokens the monolingual index cannot match.

10.5 LaBSE-Cross vs LaBSE-Mono

LaBSE-Cross (MAP = 0.298) trails LaBSE-Mono (MAP = 0.387) for two reasons:

- Larger candidate pool — cross-lingual search spans all 400K documents (4x more than mono), increasing competition from semantically similar documents in other languages.
- Despite this, cross-lingual retrieval places the relevant document in the top-10 for 56.7% of queries — a strong result that monolingual VSM cannot replicate across language boundaries.

11. How Text Mining Techniques Improve Web Search

While the quantitative evaluation shows mixed results for query expansion on this specific dataset, the four text mining techniques each contribute to improving web search in complementary ways.

11.1 Document Clustering for Topical Navigation

The 12 KMeans+LSA clusters enable multiple improvements to search:

- Faceted navigation — cluster labels (Sports, Politics, Cinema) let users refine intent after submitting a query.
- Corpus overview — visualisations reveal what topics are covered and how documents relate to each other.
- Query routing — clusters can direct queries to specialised backends (e.g., a sports API for cluster 2).

11.2 Topic Modelling for Query Understanding

LDA topic distributions improve handling of ambiguous queries:

- Disambiguation — a query like "java" can be resolved by comparing its topic distribution against known topic profiles (programming vs. geography).
- Grouped results — results can be presented by topic, letting the user choose the intended sense.
- Intent inference — session-level topic transitions reveal how user intent evolves across queries.

11.3 Query Expansion for Vocabulary Coverage

Query expansion targets the vocabulary mismatch problem — a user searches "car" while the document says "automobile." The cluster-based approach:

- Broadens the query's vocabulary footprint by injecting topically related terms from the nearest cluster centroid.
- Improved Spanish MAP from 0.014 to 0.029 (2x gain), showing promise when cluster-language alignment holds.
- Would yield stronger overall gains with multilingual clusters or relevance-feedback-based term selection.

11.4 Keyphrase Extraction for Snippet Quality

Per-document keyphrases improve the search experience at multiple touchpoints:

- Snippet enrichment — keyphrases anchor sentence selection for more relevant result previews.
- Result cards — keyphrases appear alongside results in the Streamlit app, giving users content signals before clicking through.
- Query features — keyphrases can power auto-complete suggestions and "related searches" panels.

11.5 Dense Embeddings as the Future of Web Search

LaBSE dense retrieval outperforms TF-IDF by 2.5x on MAP. Dense embeddings provide three capabilities that sparse matching fundamentally cannot:

- Semantic similarity — paraphrases ("car" vs "automobile") and synonyms are matched without exact token overlap.
- Cross-lingual alignment — semantically equivalent queries and documents in different languages are mapped to the same vector space.
- Hybrid architecture readiness — sparse retrieval handles efficiency and exact-match; dense re-ranking adds semantic understanding. This system demonstrates that paradigm at scale.

12. Conclusions & Future Work

12.1 Performance Summary

Table 10: Summary of method strengths and weaknesses.

Method	Strength	Weakness
VSM	Fast, interpretable, strong for exact-match queries	Vocabulary mismatch across languages; no semantic similarity
VSM+QE	Can improve recall by broadening query vocabulary	Expansion quality depends on cluster coherence; may add noise
LaBSE-Mono	Captures semantic similarity; consistent across EN/ES/FR/DE	Slower inference; requires pre-computed document embeddings
LaBSE-Cross	True cross-lingual retrieval — query in any language retrieves across all 4	Slightly lower precision due to 4x candidate pool

12.2 Key Findings

- Finding 1: LaBSE consistently outperforms VSM on MAP because it captures paraphrase and semantic equivalence that exact-token matching misses. The gap is especially pronounced for French and Spanish, where stemmed surface forms fail to match.
- Finding 2: Query expansion via cluster terms can improve or degrade VSM depending on cluster coherence and language alignment. English-only clusters injecting terms into non-English queries is counterproductive.
- Finding 3: Cross-lingual LaBSE is competitive with monolingual LaBSE and uniquely capable of retrieving relevant documents in different languages from any query language.
- Finding 4: Title-based evaluation queries with guaranteed ground-truth provide a sound, reproducible evaluation methodology when official qrels may not overlap with a corpus subset.

12.3 Limitations & Future Work

- Clustering and LDA are fitted on English only; extending to multilingual text mining would improve non-English query expansion.
- Evaluation uses single-relevant-document queries; real IR evaluation typically has multiple judged documents per query.
- Hybrid retrieval (sparse + dense re-ranking) would likely outperform either method alone.

13. Assumptions

Table 11: Assumptions made in the implementation.

#	Assumption	Rationale
1	All metrics computed at rank cutoff k=10	Standard IR practice; top-10 retrieval is the target evaluation depth
2	LaBSE embeddings are float32, L2-normalised	Cosine similarity reduces to dot product — faster computation
3	Random seed 42 used throughout	Reproducibility of results
4	Queries with zero relevant results produce AP=0	Standard IR convention; included in MAP averaging
5	Cross-lingual uses shared embedding (LaBSE) instead of query translation	More robust than dictionary-based; handles 109 languages natively

14. References

[1] Zhang, X., et al. (2023). MIRACL: A Multilingual Retrieval Dataset Covering 18 Languages. TACL.

[2] Feng, F., et al. (2022). Language-agnostic BERT Sentence Embedding (LaBSE). ACL.

[3] Manning, C.D., Raghavan, P., & Schutze, H. (2008). Introduction to Information Retrieval. Cambridge University Press.

[4] Blei, D.M., Ng, A.Y., & Jordan, M.I. (2003). Latent Dirichlet Allocation. JMLR, 3, 993-1022.

[5] Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management.

[6] HuggingFace Dataset: https://huggingface.co/datasets/thoshiths/miracl-multilingual-4M

[7] HuggingFace Spaces (Live Demo): https://huggingface.co/spaces/thoshiths/miracl-search

[8] Source Code: https://github.com/thoshith-s/wilp-sem2-ir-assignment-2